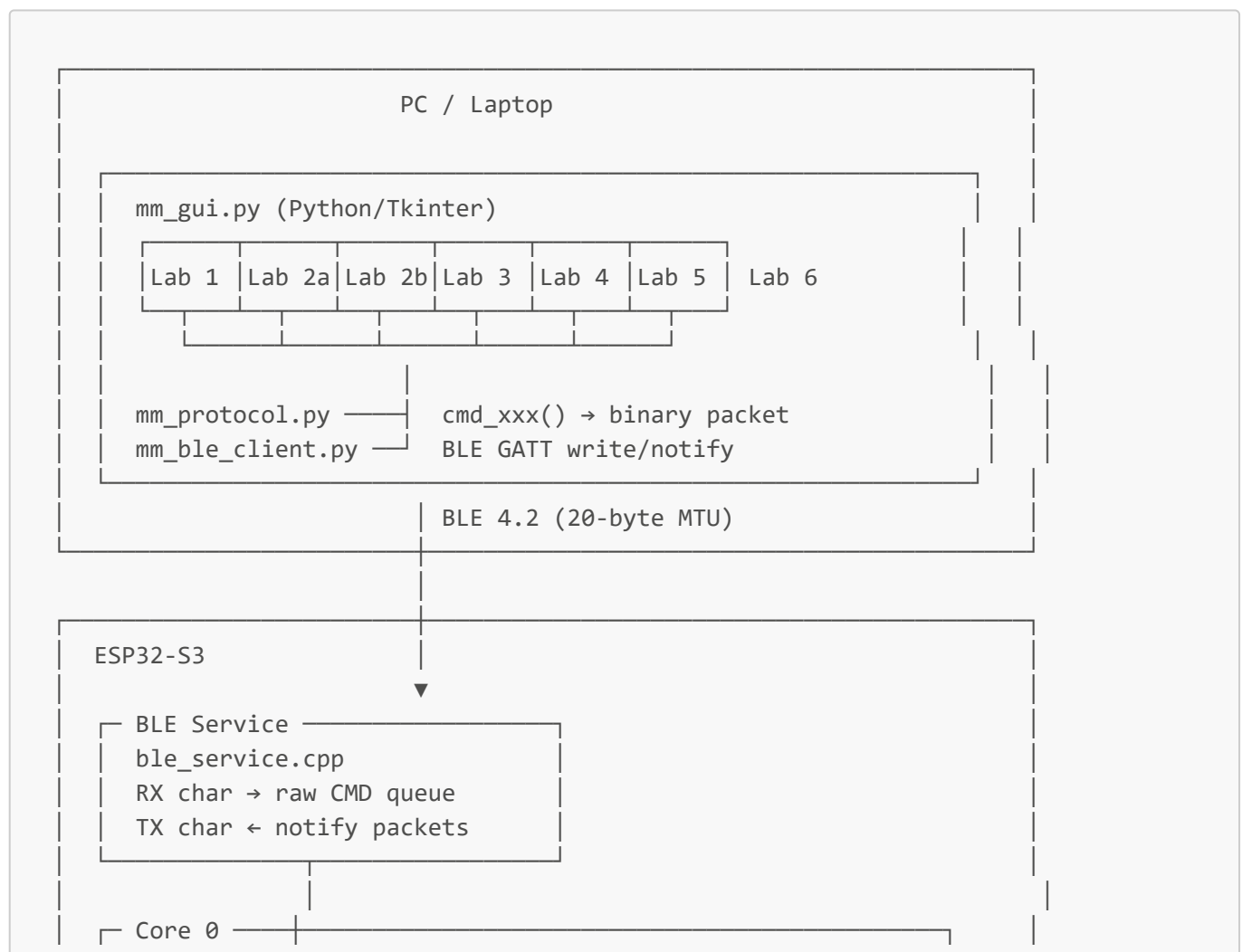


Micromouse ESP32-S3 — System Architecture

สารบัญ

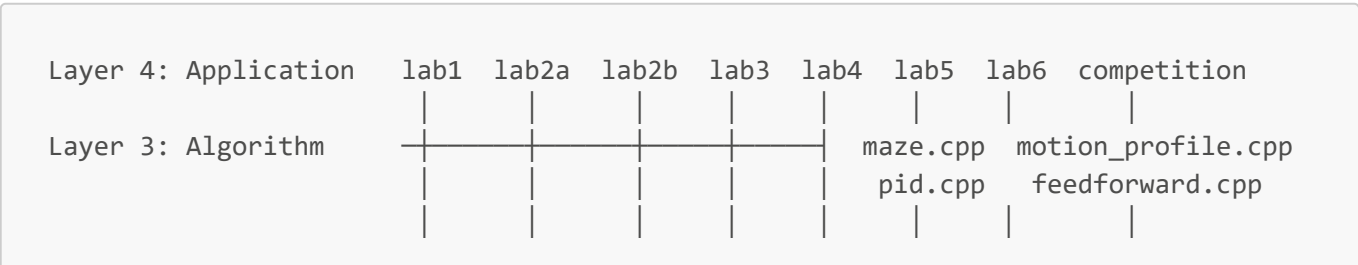
1. ภาพรวมระบบ
2. สถาปัตยกรรม Hardware/Software Layers
3. FreeRTOS Tasks & Timing
4. BLE Protocol & Packet Flow
5. Lab 1 — Hardware Check & Calibration
6. Lab 2a — Encoder Calibration
7. Lab 2b — Motor Characterization
8. Lab 3 — Motion Profile (Open-Loop)
9. Lab 4 — PID Closed-Loop Control
10. Lab 5 — Maze Search (Flood Fill)
11. Lab 6 — Speed Run
12. GUI (mm_gui.py) — Tab Structure
13. Cross-Lab Dependencies
14. Config.h Parameter Map

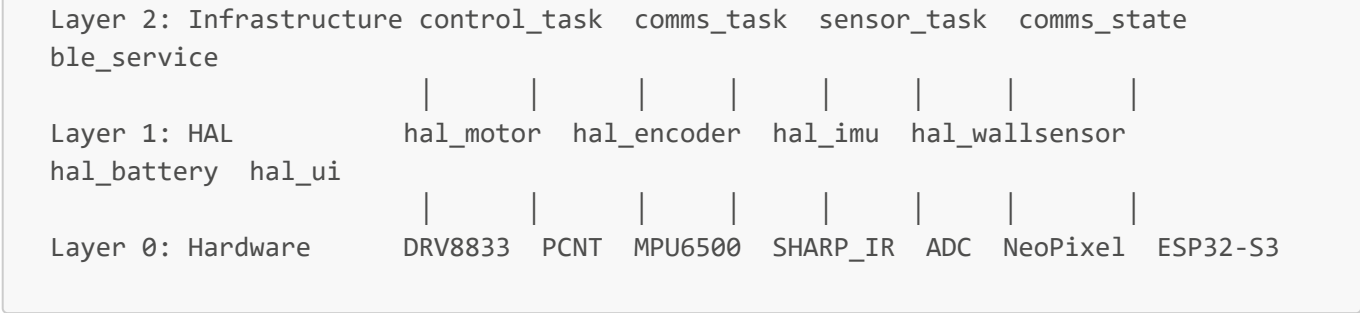
1. ภาพรวมระบบ





2. สถาปัตยกรรม



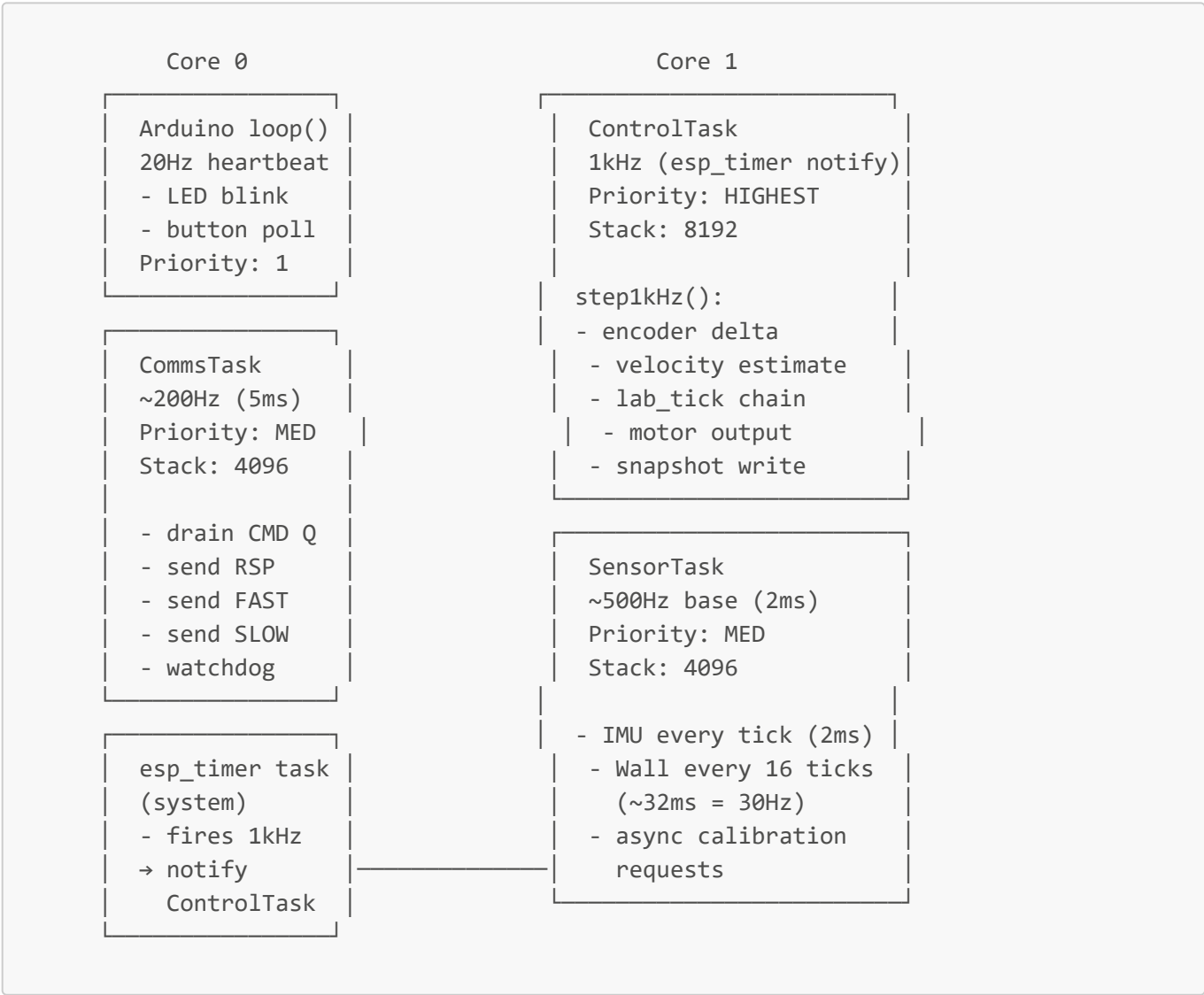


ไฟล์ทั้งหมด:

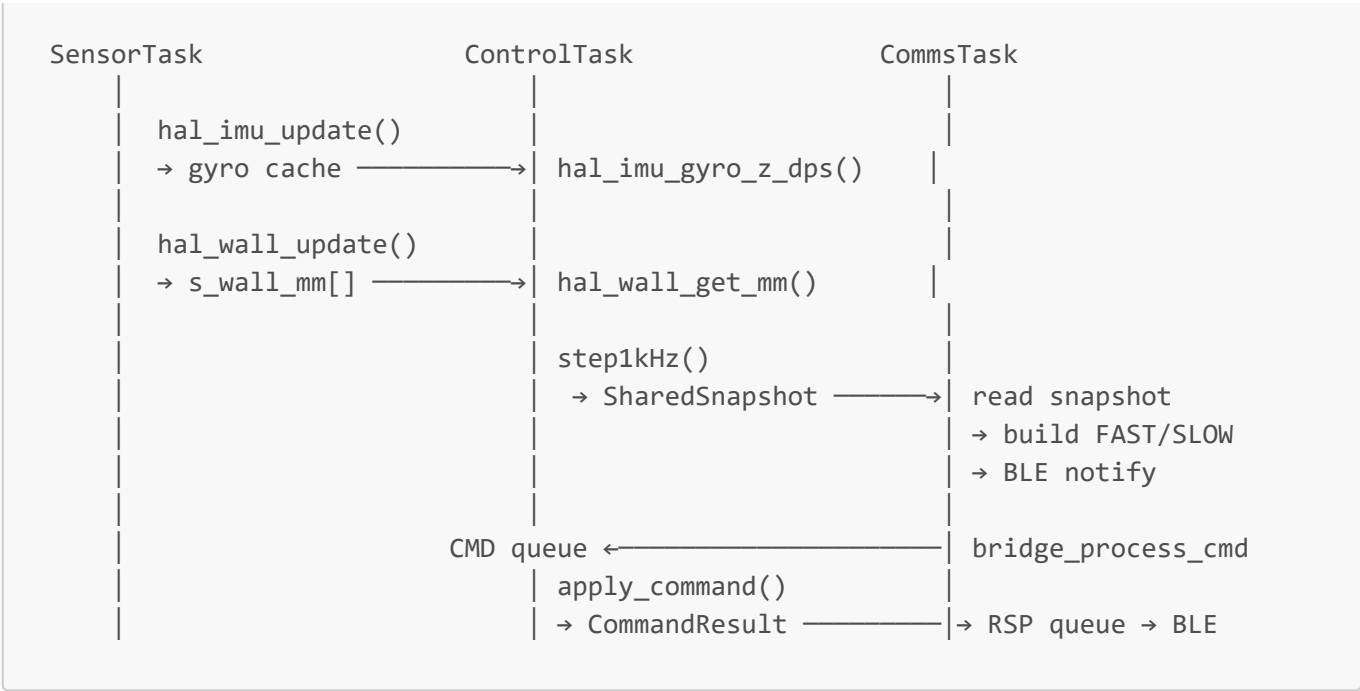
ไฟล์	Layer	หน้าที่
micromouse_esp32s3.ino	Main	setup() init ทุกอย่าง, loop() heartbeat LED only
config.h	Config	#define ทุกค่าที่ tune ได้ (pin, threshold, PID, speed)
protocol.h	Protocol	packet format, CMD/RSP opcode, test_id, CRC
hal_motor.cpp/h	HAL	PWM → mV → duty, brake/coast/set_mV
hal_encoder.cpp/h	HAL	PCNT quadrature, delta_L/R per tick
hal_imu.cpp/h	HAL	SPI gyro/accel, calibrate, gyro_z_dps
hal_wallsensor.cpp/h	HAL	IR ADC→mm / VL6180X I2C, multi-sensor manager
hal_battery.cpp/h	HAL	voltage divider ADC, level/critical check
hal_ui.cpp/h	HAL	NeoPixel LED, buttons, DIP switch
hal_buzzer.cpp/h	HAL	non-blocking beep/melody
sensor_task.cpp/h	Infra	FreeRTOS task: IMU 500Hz + wall 30Hz
control_task.cpp/h	Infra	1kHz tick: cmd dispatch + lab chain + motor output
comms_task.cpp/h	Infra	BLE packet build/send: FAST/SLOW/RSP
comms_state.cpp/h	Infra	SharedSnapshot, CMD/RSP queues, portMUX
ble_service.cpp/h	Infra	BLE GATT server, RX/TX characteristics
telemetry_bridge.cpp/h	Infra	CMD validation, lab5 query shortcut
motion_profile.cpp/h	Algo	trapezoidal/S-curve velocity profile
pid.cpp/h	Algo	generic PD/PID controller
feedforward.cpp/h	Algo	motor voltage model: bias + speed×Kv + accel×Ka
maze.cpp/h	Algo	16×16 maze struct, BFS flood fill, wall ops
lab1_hw_check.cpp/h	App	hardware test: motor/LED/buzzer/sensor/cal
lab2a_enc_cal.cpp/h	App	encoder CPR measurement
lab2b_motor_char.cpp/h	App	step response, ramp, bias finding

ไฟล์	Layer	หน้าที่
lab3_motion.cpp/h	App	open-loop trapezoidal profile drive/turn
lab4_pid.cpp/h	App	closed-loop PID: straight/turn/arc
lab5_search.cpp/h	App	maze search: flood fill + wall detect + safety
lab6_speedrun.cpp/h	App	speed run: fast execution of known path
competition.cpp/h	App	competition mode: search → optimize → fast run
mm_gui.py	PC	Tkinter GUI with tab per lab
mm_protocol.py	PC	binary packet builder/parser
mm_ble_client.py	PC	BLE GATT client (bleak library)
mm_run_analyzer.py	PC	post-run data analysis + CSV export

3. FreeRTOS Tasks



Data flow ระหว่าง tasks:



4. BLE Protocol & Packet Flow

Packet Types

GUI → Firmware:

CmdPacket (PKT_CMD = 0x20) 20 bytes

[type][seq][opcode][req_id×2][payload×14]

[crc]

Firmware → GUI:

RspPacket (PKT_RSP = 0x30) 20 bytes

[type][seq][opcode][req_id×2][status]

[payload×13][crc]

FastPacket (PKT_FAST = 0x01) 20 bytes

t_ms, vCmdL, vCmdR, velL, velR,

v_target, gyroZ, encDelta, state

→ ส่งที่ 20-50Hz (ปรับได้)

SlowPacket (PKT_SLOW = 0x02) 20 bytes

vBat, missed_ticks, step_max_us,

active_flags, wallMM[4], wallBitmap

→ ส่งที่ 2-5Hz

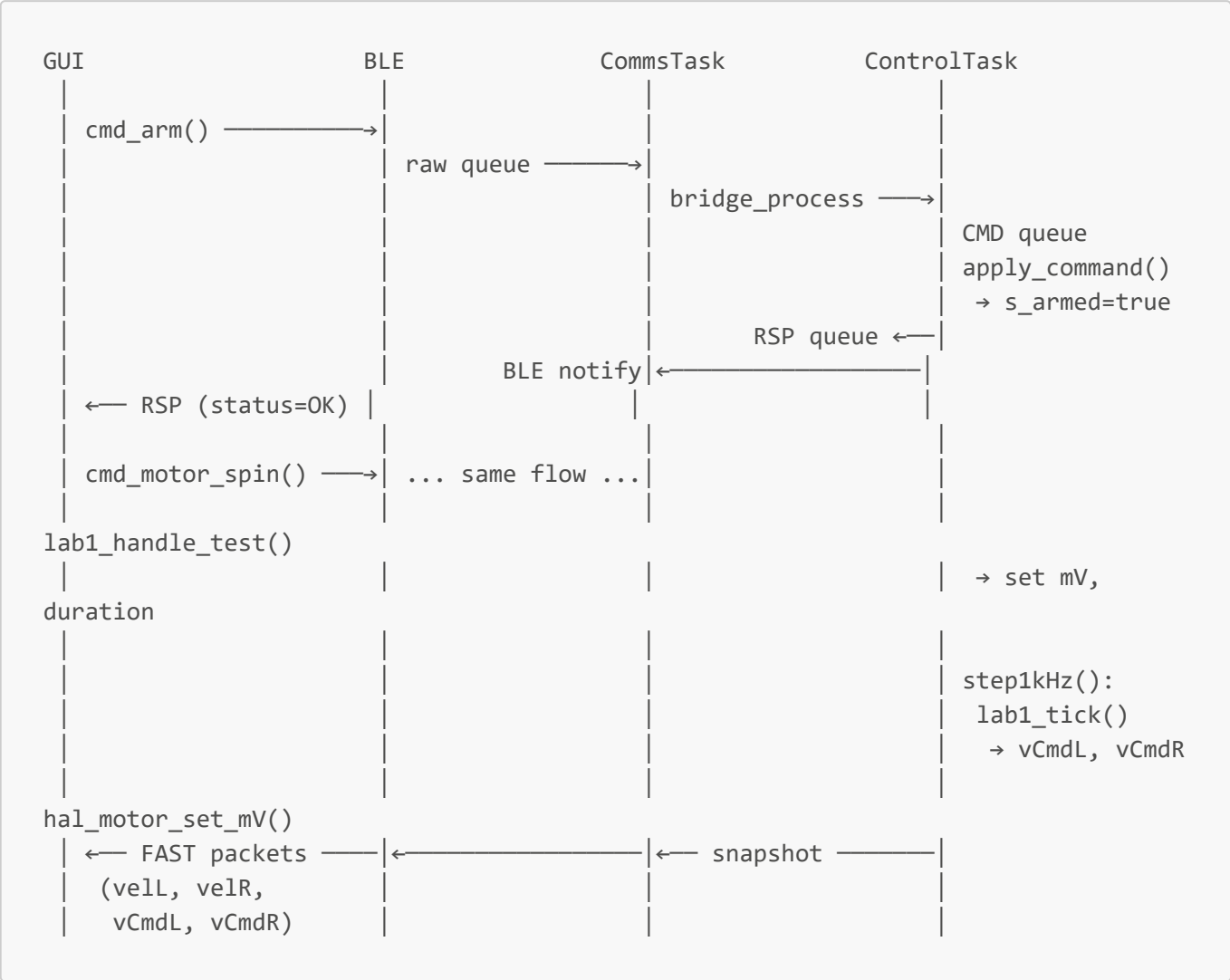
CapsPacket (PKT_CAPS = 0x10) 20 bytes

firmware version, sensor count/type,

maze size, feature bits

→ ส่งครั้งเดียวตอน connect

Command Flow (ตัวอย่าง ARM → Motor Spin)



5. Lab 1 — Hardware Check & Calibration

วัตถุประสงค์

ทดสอบ hardware ทุกชิ้น: motor, LED, buzzer, sensor, IMU, battery + calibrate gyro/wall sensor

Test IDs & Commands

Test ID	ชื่อ	GUI Button	พารามิเตอร์	ต้อง ARM?
0x01	MOTOR_SPIN	Motor L/R/Both	mV, duration_ms, select	✓
0x02	LED_SET	LED color	R, G, B	✓
0x03	BUZZER	Buzzer	freq_hz, duration_ms	✓

Test ID	ชื่อ	GUI Button	พารามิเตอร์	ต้อง ARM?
0x04	READ_SENSORS	Read Walls	-	X (read-only)
0x05	READ_BATTERY	Read Battery	-	X
0x06	READ_IMU	Read IMU	-	X
0x07	READ_BUTTONS	Read Buttons	-	X
0x08	LED_BRIGHTNESS	LED Brightness	brightness	✓
0x09	CAL_GYRO	Calibrate Gyro	samples÷10	X
0x0A	CAL_WALLS	Calibrate Walls	samples÷10	X
0x0B	CAL_QUERY	Query Cal	-	X
0x0C	WALL_SET_OFFSET	Set Offset	position, offset_mm	X

Firmware Flow

```
lab1_handle_test(cmd):
    switch test_id:
        MOTOR_SPIN → set s_mV, s_dur, s_side → start tick countdown
        LED_SET → hal_ui_led_set(r,g,b) → immediate
        READ_SENSORS → hal_wall_get_all() → RSP payload [L,FL,FR,R] mm
        CAL_GYRO → sensor_request_gyro_cal(N) → async, status polled
        CAL_WALLS → sensor_request_wall_cal(N) → async, status polled

lab1_tick(vCmdL, vCmdR):
    if running:
        *vCmdL = s_mV (or 0 if side=right-only)
        *vCmdR = s_mV (or 0 if side=left-only)
        if tick_count >= s_dur → stop, return false
    return still_running
```

GUI (Lab1Tab)

Lab 1: Hardware Check

[ARM] [DISARM] [STOP] [ABORT]

— Motor —

mV: [__] dur: [__]ms

[Motor L] [Motor R] [Both]

— Calibration —

[Cal Gyro] status: done

[Cal Walls] status: idle

[Query Cal] [Apply]

— LED / Buzzer —

[LED] R[___] G[___] B[___]

[Buzzer] freq[___] dur[___]

— Sensors —

[Read Walls] L:35 FL:26

[Read Battery] 7.82V

[Read IMU] gz: -0.3°/s

```
GUI → CAL_WALLS → firmware → sensor_request_wall_cal(50)
                                → SensorTask: hal_wall_calibrate(50) ≈ 1.25s
                                → status = 2 (done)
GUI polls CAL_QUERY → RSP: raw averages + current offsets
GUI shows offsets → user clicks "Apply" → WALL_SET OFFSET per sensor
```

- **Gyro calibration** → ใช้ทุก lab ที่ต้องการ heading (Lab 4, 5, 6)
- **Wall calibration** → ใช้ Lab 5 wall detection
- **Motor test** → ยืนยันทิศทางถูกต้องก่อนไป Lab 2b
- **Battery read** → config **BATTERY_DIVIDER_RATIO**

วัดสุประสงค์

วัดค่า CPR (Counts Per Revolution) ของ encoder เพื่อคำนวณ **MM PER COUNT**

Test ID	ชื่อ	หน้าที่
0x10	ENC_MONITOR	อ่านค่า encoder สะสม (absolute counts)
0x11	ENC RESET	reset encoder accumulators = 0

```
lab2a_handle_test(cmd):
    ENC_MONITOR → RSP: countL (i32), countR (i32)
    ENC RESET   → hal encoder reset both()
```

```
Lab 2a: Encoder Calibration
```

ขั้นตอน:

1. [Reset] encoder counts = 0
2. หมุนล้อด้วยมือ 1 รอบ (ใช้ mark บนล้อ)
3. [Read] encoder counts

Left: 815 counts Right: 818 counts


```
Wheel diameter: [33.5] mm
[Calculate]
CPR = 815 (ใช้ค่าเฉลี่ย)
MM_PER_COUNT =  $\pi \times 33.5 / 815 = 0.1291$  mm/count
DEG_PER_COUNT =  $360 / 815 = 0.4417$  °/count

→ ใส่ค่าลง config.h: ENCODER_CPR, MM_PER_COUNT
```

Lab 2a เชื่อมต่อ Lab อื่น

- ค่า `MM_PER_COUNT`, `DEG_PER_COUNT` → ใช้ทุก Lab ที่มี velocity estimation (3,4,5,6)

7. Lab 2b — Motor Characterization

วัตถุประสงค์

หา motor parameters: bias voltage, speed-voltage curve, step response → สร้าง feedforward model

Test IDs

Test ID	ชื่อ	พารามิเตอร์	หน้าที่
0x20	RAMP_UP	start_mV, end_mV, step_mV, step_ms	ค่อย ๆ เพิ่มแรงดัน หา no-load threshold
0x21	STEP_RESPONSE	mV, duration_ms	จ่าย voltage คงที่ วัด velocity response
0x22	FIND_BIAS	-	auto-find minimum mV ให้ล้อหมุน

Firmware Flow

```
lab2b_handle_test(cmd):
    RAMP_UP → set start/end/step/timing → tick ramps voltage
    STEP_RESPONSE → set mV + duration → tick holds constant
    FIND_BIAS → auto ramp each wheel → detect motion → RSP: biasL, biasR

lab2b_tick(vCmdL, vCmdR):
    if RAMP mode:
        if tick_in_step >= step_ms → increase by step_mV
        if current_mV >= end_mV → done
    if STEP mode:
        hold mV for duration → done
    if BIAS mode:
        ramp slowly → detect encoder movement → record bias → switch wheel → done
```

GUI (Lab2bTab)

```
Lab 2b: Motor Characterization
[ARM] [DISARM]

— Ramp Up —           — Step Response —
start[0] end[3000] step[100] mV[2000] dur[2000]ms
delay[200]ms           [Step L] [Step R] [Both]
[Ramp L] [Ramp R] [Both]

[Find Bias] → biasL=570 biasR=590 (auto)

Velocity Plot
vel (mm/s) vs time
████████████████████████████████████████████████████████████████████████████████

Command Plot
mV vs time
████████████████████████████████████████████████████████████████████████████████

→ ผลลัพธ์ใส่ config.h:
FF_LINEAR_BIAS_MV, FF_LINEAR_KV, FF_LINEAR_KA
```

Lab 2b เชื่อมต่อ Lab อื่น

- **Bias** → `feedforward.cpp` `FF_LINEAR_BIAS_MV` (ค่า mV ต่ำสุดที่มอเตอร์หมุน)
- **Speed-voltage curve** → `FF_LINEAR_KV` (mV per mm/s)
- **Step response** → `FF_LINEAR_KA` (mV per mm/s² acceleration)
- ค่าทั้งหมด → ใช้ใน Lab 3 open-loop และ Lab 4+ closed-loop

8. Lab 3 — Motion Profile (Open-Loop)

วัตถุประสงค์

ทดสอบ trapezoidal velocity profile + feedforward → รังตรง/หมุนแบบ open-loop

Test IDs

Test ID	ชื่อ	พารามิเตอร์	หน้าที่
0x30	TRAP_FWD	distance_mm, max_vel, accel	รังตรง trapezoidal
0x31	TRAP_TURN	angle_deg, max_dps, accel	หมุน trapezoidal
0x32	SCURVE_FWD	distance, vel, accel, jerk	รังตรง S-curve (ถ้ามี)

Firmware Flow

```
lab3_handle_test(cmd):
    TRAP_FWD → profile_init(&prof, distance_mm, vel, accel, decel)
    s_mode = FWD
```

```

TRAP_TURN → profile_init(&prof, angle_deg, dps, accel, decel)
                s_mode = TURN

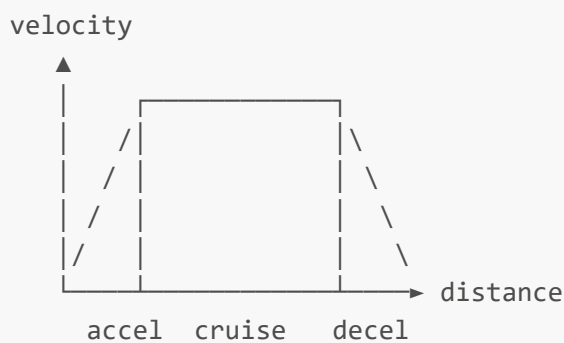
ab3_tick(vCmdL, vCmdR, dL, dR):
    // track actual distance/angle from encoder deltas
    actual_dist += (dL + dR) × MM_PER_COUNT / 2
    target_vel = profile_tick(&prof, actual_dist)
    target_acc = profile_get_accel(&prof)

    if FWD:
        ff_L = ff_voltage_L(target_vel, target_acc)
        ff_R = ff_voltage_R(target_vel, target_acc)
    if TURN:
        v_wheel = w × trackWidth/2
        ff_L = ff_voltage_L(+v_wheel, ...)
        ff_R = ff_voltage_R(-v_wheel, ...)

    *vCmdL = ff_L
    *vCmdR = ff_R
    return !profile_is_done

```

Motion Profile (trapezoidal)



GUI (Lab3Tab)

```
Lab 3: Motion Profile
[ARM] [DISARM] [STOP]

dist[180]mm vel[200]mm/s accel[2000]mm/s2
[Forward] [Backward]

angle[90]° vel[200]°/s accel[500]°/s2
[Turn CW] [Turn CCW]

Velocity Plot
target vs actual velocity

→ ดูว่า actual ตาม target ไหม
```

→ ถ้า drift → ต้อง tune FF gains → ไป Lab 4

Lab 3 เชื่อมต่อ Lab อื่น

- ใช้ `motion_profile.cpp` + `feedforward.cpp` จาก Lab 2b
- ถ้า open-loop ไม่ตรง → ต้องเพิ่ม PID → Lab 4
- Profile structure เดียวกันใช้ต่อใน Lab 4, 5, 6

9. Lab 4 — PID Closed-Loop Control

วัตถุประสงค์

เพิ่ม PID feedback: velocity PID + heading PID + wall centering → รีดตรง/หมุนแม่นยำ

Test IDs

Test ID	ชื่อ	หน้าที่
0x40	STRAIGHT	รีดตรง N mm ด้วย velocity PID + heading PID
0x41	TURN_90	หมุน 90° ด้วย heading PID
0x42	TURN_180	หมุน 180°
0x43	SMOOTH_ARC	arc turn (radius + angle)
0x44	MOVE_CELLS	รีด N cells (180mm × N)

Firmware Flow

```
lab4_handle_test(cmd):
    STRAIGHT → profile_init(distance, vel, accel, decel)
                pid_reset(vel_pid, heading_pid)
                s_mode = STRAIGHT

lab4_tick(vCmdL, vCmdR, velL, velR, gyroZ):
    // Integrate gyro for heading
    actual_heading += gyroZ × GYRO_SIGN × dt

    // Velocity PID
    actual_dist += |avg_vel| × dt
    target_vel = profile_tick(&prof, actual_dist)
    vel_err = target_vel - avg_vel
    vel_corr = pid_tick(&vel_pid, vel_err)

    // Heading PID (keep straight)
    heading_err = target_heading - actual_heading
    heading_corr = pid_tick(&heading_pid, heading_err)

    // Feedforward + PID corrections
```

```

ff_L = ff_voltage_L(target_vel, target_acc)
ff_R = ff_voltage_R(target_vel, target_acc)

*vCmdL = ff_L + vel_corr - heading_corr
*vCmdR = ff_R + vel_corr + heading_corr

```

PID Tuning Commands (realtime จาก GUI)

CMD	หน้าที่
CMD_SET_PD_LINEAR (0x42)	Kp, Kd สำหรับ velocity PID
CMD_SET_PD_STEERING (0x44)	Kp, Ki, Kd สำหรับ heading PID + steer trim
CMD_SET_FF_LINEAR (0x40)	bias, Kv, Ka สำหรับ feedforward

GUI (Lab4Tab)

Lab 4: PID Tuning

[ARM] [DISARM] [STOP]

— Linear PID —

Kp[0.5] Kd[0.1]

[Send Linear]

— Steering PID —

Kp[2.0] Ki[0] Kd[0.5]

[Send Steering]

— Feedforward —

biasL[570] biasR[590]

Kv[1.0] Ka[0.5]

[Send FF]

— Commands —

dist[180]mm vel[200]

[Fwd] [Back] [CW] [CCW]

[Move 1 Cell] [Move N Cells]

— Steer Trim —

trim[-50..+50]mV [Apply]

Velocity Plot (target vs actual)

Heading Plot (target vs actual)

Motor Cmd Plot (L vs R mV)

[Export CSV] [Clear]

Lab 4 เชื่อมต่อ Lab อื่น

- ใช้ `feedforward.cpp` จาก Lab 2b/3
- ใช้ `motion_profile.cpp` จาก Lab 3
- เพิ่ม `pid.cpp` (velocity + heading)
- ค่า PID ที่ tune ได้ → ใช้ต่อใน Lab 5, 6 โดยตรง
- `lab5_search.cpp` เรียก `pid_tick()` + `profile_tick()` + `ff_voltage_L/R()` เหมือน Lab 4

10. Lab 5 — Maze Search (Flood Fill)

วัตถุประสงค์

หุ่นยนต์ค้นหาเส้นทางใน maze อัตโนมัติด้วย flood fill algorithm + wall detection

Test IDs

Test ID	ชื่อ	หน้าที่	ARM?
0x50	SEARCH_RUN	วิ่ง search ต่อเนื่อง/step-by-step	✓
0x51	SEARCH_STEP	เดิน 1 cell (debug ทีละ step)	✓
0x52	SEARCH_RETURN	กลับ (0,0) ตาม shortest path	✓
0x53	SEARCH_RESET	reset maze + position	✓
0x54	READ_WALLS	อ่าน wall sensor mm (debug)	✗
0x55	SEARCH_QUERY	อ่าน pose + state (GUI poll)	✗
0x56	MAZE_ROW	อ่าน wall data 1 row (GUI poll)	✗
0x57	WALL_FOLLOW_L	wall follow ซ้าย (continuous)	✓
0x58	WALL_FOLLOW_R	wall follow ขวา (continuous)	✓

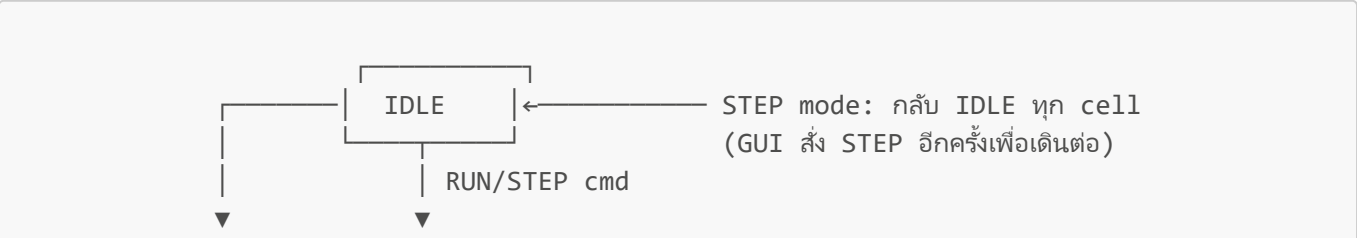
Inline Config (★ หลีกเลี่ยง firmware version dependency)

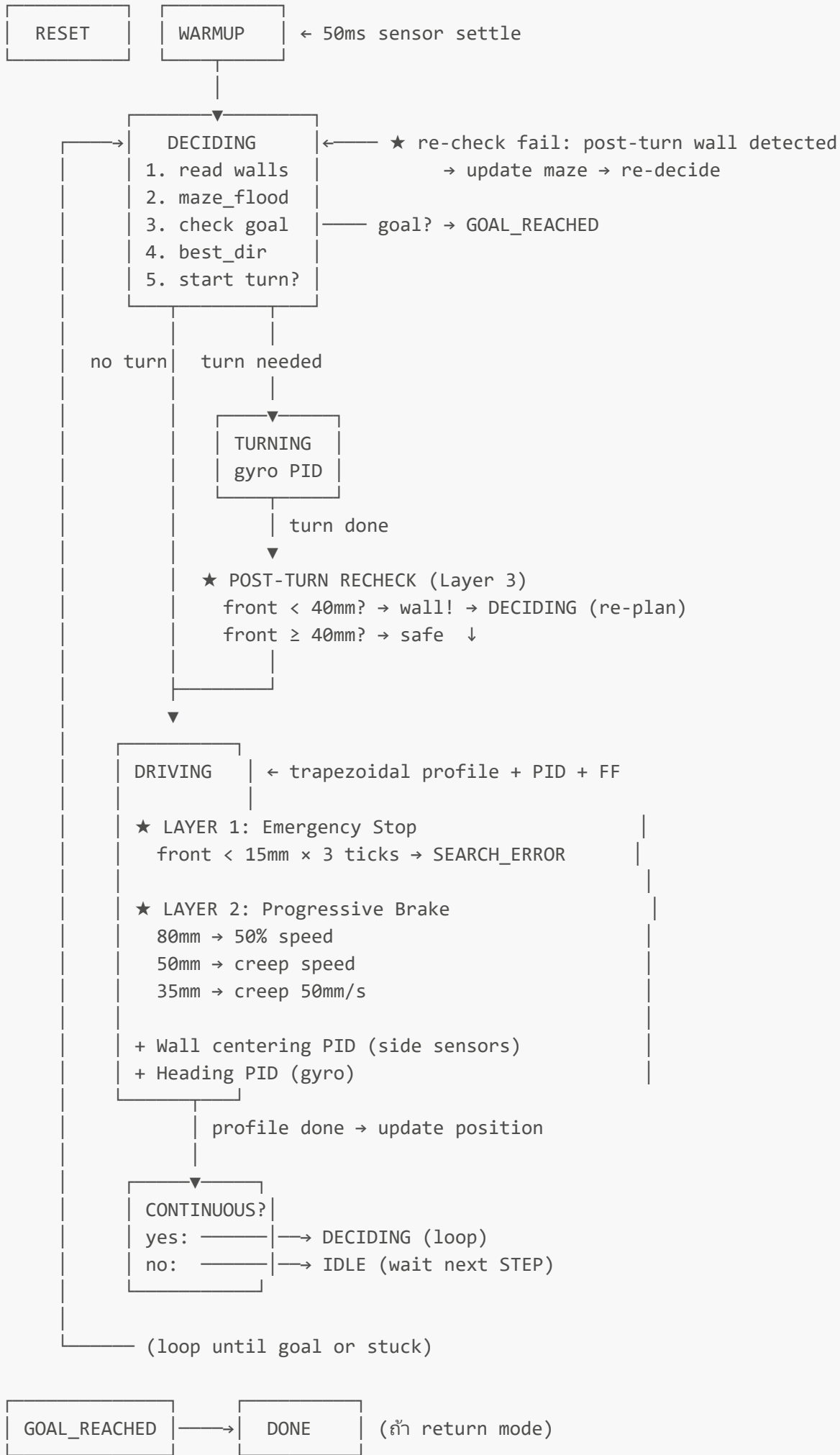
RUN/STEP/RESET command payload:

- [0] test_id
- [1] mode (RUN only)
- [2] 0xCF = magic marker (config present)
- [3] start_x
- [4] start_y
- [5] start_heading (0=N,1=E,2=S,3=W)
- [6] goal_x_min
- [7] goal_y_min
- [8] goal_x_max
- [9] goal_y_max

→ firmware ตรวจสอบ byte[2]==0xCF → อ่าน config → ไม่ต้องฟัง 0x59 handler

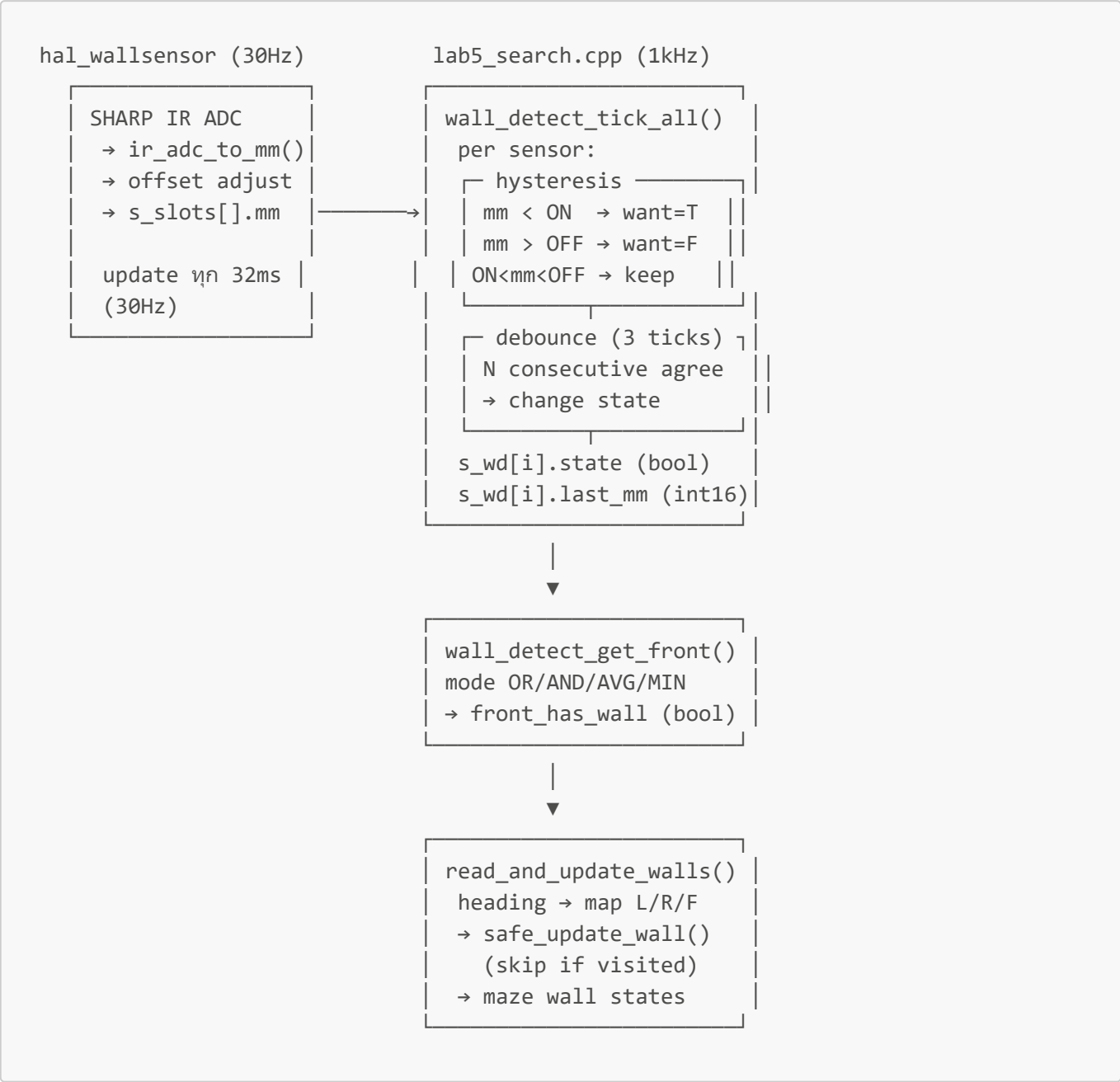
State Machine



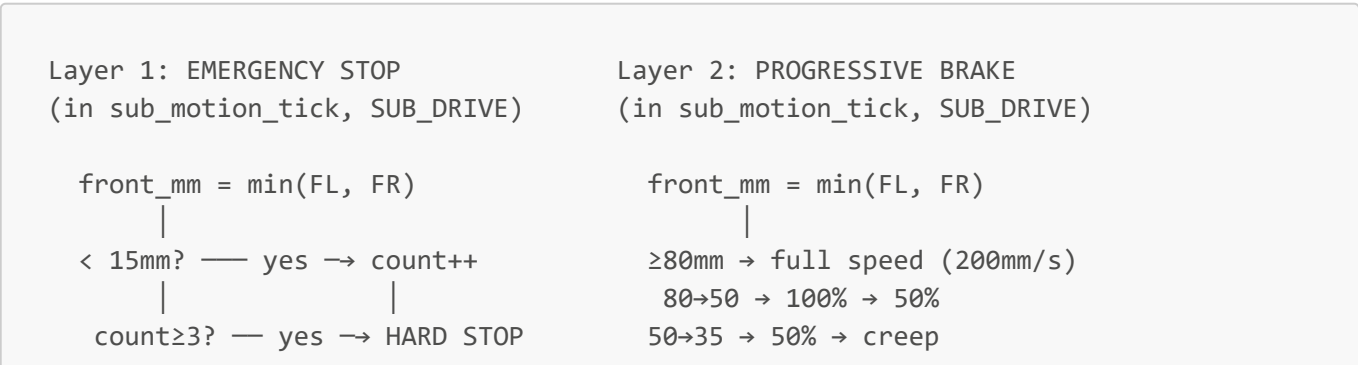




Wall Detection System (1kHz tick)



Front Safety System (3 Layers)




```
front = min(FL, FR)
|
< 40mm? — yes → wall detected!
|               safe_update_wall(heading, WALL)
|               → DECIDING (re-plan, don't drive)
no
└→ start_drive() (normal)
```

Maze & Flood Fill

```

maze.cpp:
Maze struct: 16x16 cells
    walls[y][x]    - 4-bit wall bitmap (N,E,S,W)
    visited[y][x]  - has robot been here?
    cost[y][x]     - BFS distance to goal

maze_flood(maze):
    BFS from goal cells
    expand to neighbors where wall = EXIT or UNKNOWN
    cost = neighbor + 1

maze_best_dir(maze, x, y, heading):
    check 4 neighbors
    pick lowest cost that is passable
    prefer current heading (reduce turns)

Wall states per edge: UNKNOWN / WALL / EXIT
    UNKNOWN → flood fill treats as passable (optimistic)
    WALL    → flood fill blocks
    EXIT    → flood fill allows

```

GUI (Lab5Tab) — Maze Visualization + Control

```
| Lab 5: Maze Search |-----|
```

```
| [ARM] [DISARM]
```

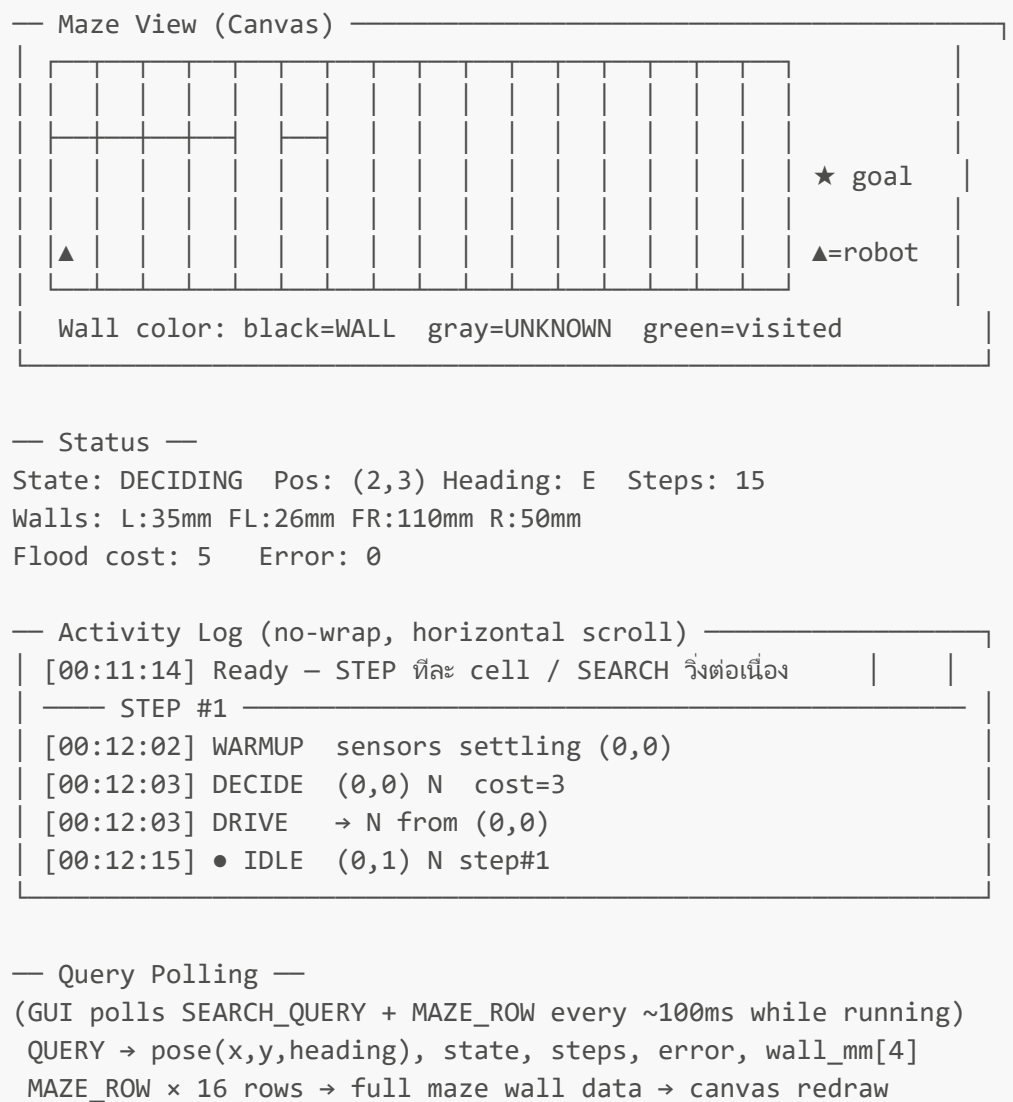
```
|
```

```
| --- Config ---                |--- Controls ---|
```

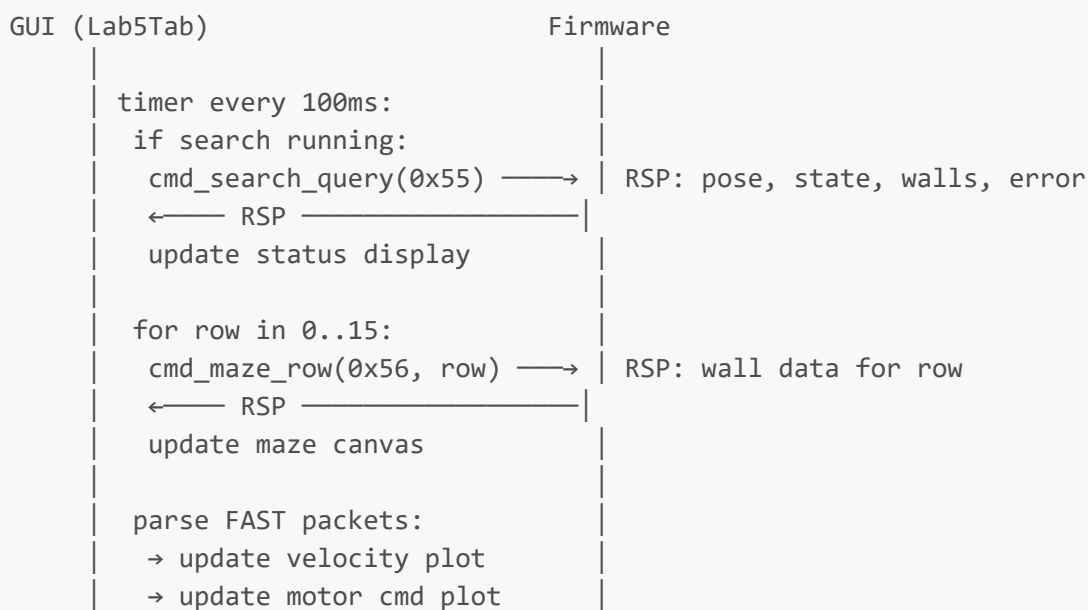
```
| Start: x[0] y[0] heading[N▼] | [SEARCH cont]  [SEARCH S&G]|
```

```
| Goal:   x_min[7] y_min[7]     |[STEP]          [RESET]    |
```

```
|         x_max[8] y_max[8]     |[RETURN]        [APPLY CONFIG]|
```



GUI Polling Loop



Lab 5 เชื่อมต่อ Lab อื่น

- **Lab 1:** wall calibration offsets → wall detection accuracy
- **Lab 2b:** feedforward model → drive/turn voltage
- **Lab 3:** motion_profile → trapezoidal drive/turn
- **Lab 4:** PID gains → velocity/heading/wall centering control
- **Lab 5 → Lab 6:** maze data (wall map) → speed run path planning

11. Lab 6 — Speed Run

วัตถุประสงค์

วิ่ง maze ที่ค้นพบแล้ว (จาก Lab 5) ด้วยความเร็วสูง + optimized path

Test IDs

Test ID	ชื่อ	หน้าที่
0x60	SPEEDRUN_START	เริ่ม speed run (payload: speed level 0-3)
0x61	SPEEDRUN_STOP	หยุด speed run
0x62	SPEEDRUN_QUERY	อ่าน state/progress (read-only)

Speed Levels

Level	Velocity	Accel	ใช้สำหรับ
0	200 mm/s	1000	test (= search speed)
1	350 mm/s	1000	moderate
2	500 mm/s	1000	fast
3	700 mm/s	1000	competition

Firmware Flow

```
lab6_handle_test(cmd):
    SPEEDRUN_START:
        1. อ่าน maze จาก lab5_get_maze()
        2. flood fill จาก start → goal
        3. trace optimal path → action sequence [FWD, FWD, TURN_R, FWD, ...]
        4. optimize: merge consecutive FWD → multi-cell straight
        5. set speed level → execute path

lab6_tick(vCmdL, vCmdR, velL, velR, gyroZ):
    execute action sequence:
        FWD_N → profile_init(N × 180mm, fast_vel, fast_accel)
        TURN → profile_init(90°, fast_turn_dps, fast_turn_accel)
```

```
PID control same as Lab 4
return !done
```

Lab 6 เชื่อมต่อ Lab อื่น

- **Lab 5 maze** → path planning input
- **Lab 4 PID** → same control loop at higher speed
- **Lab 2b FF** → feedforward at higher velocity range

12. GUI (mm_gui.py) — Tab Structure

Architecture

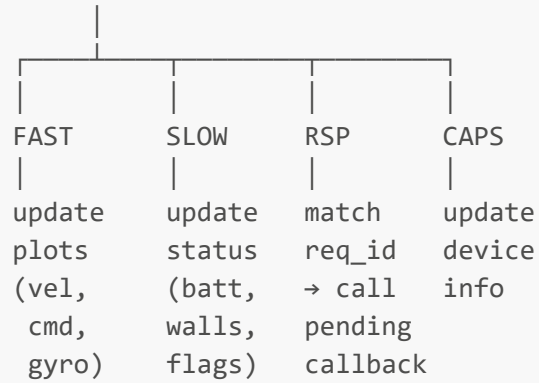
```
mm_gui.py
├── App (Tk root)
│   ├── BLE connection panel (top bar)
│   │   └── mm_ble_client.py → bleak library
│   ├── Notebook (tabs)
│   │   ├── Lab1Tab      → HW check + calibration
│   │   ├── Lab2aTab     → Encoder calibration
│   │   ├── Lab2bTab     → Motor characterization
│   │   ├── Lab3Tab      → Motion profile
│   │   ├── Lab4Tab      → PID tuning
│   │   ├── Lab5Tab      → Maze search
│   │   └── Lab6Tab      → Speed run
│   ├── Telemetry panel (bottom)
│   │   ├── FAST packet display (vel, cmd, gyro)
│   │   └── SLOW packet display (battery, wall, flags)
│   └── Status bar (connection, state, FPS)
├── mm_protocol.py
│   ├── cmd_xxx() → build CmdPacket bytes
│   ├── parse_fast() → decode FastPacket
│   ├── parse_slow() → decode SlowPacket
│   └── parse_rsp() → decode RspPacket + dispatch
└── mm_ble_client.py
    ├── scan() → find "MM_xxxx" device
    ├── connect() → GATT connect
    ├── send_cmd(bytes) → write RX characteristic
    └── on_notify(bytes) → callback for TX characteristic
```

Packet Receive Flow (GUI side)

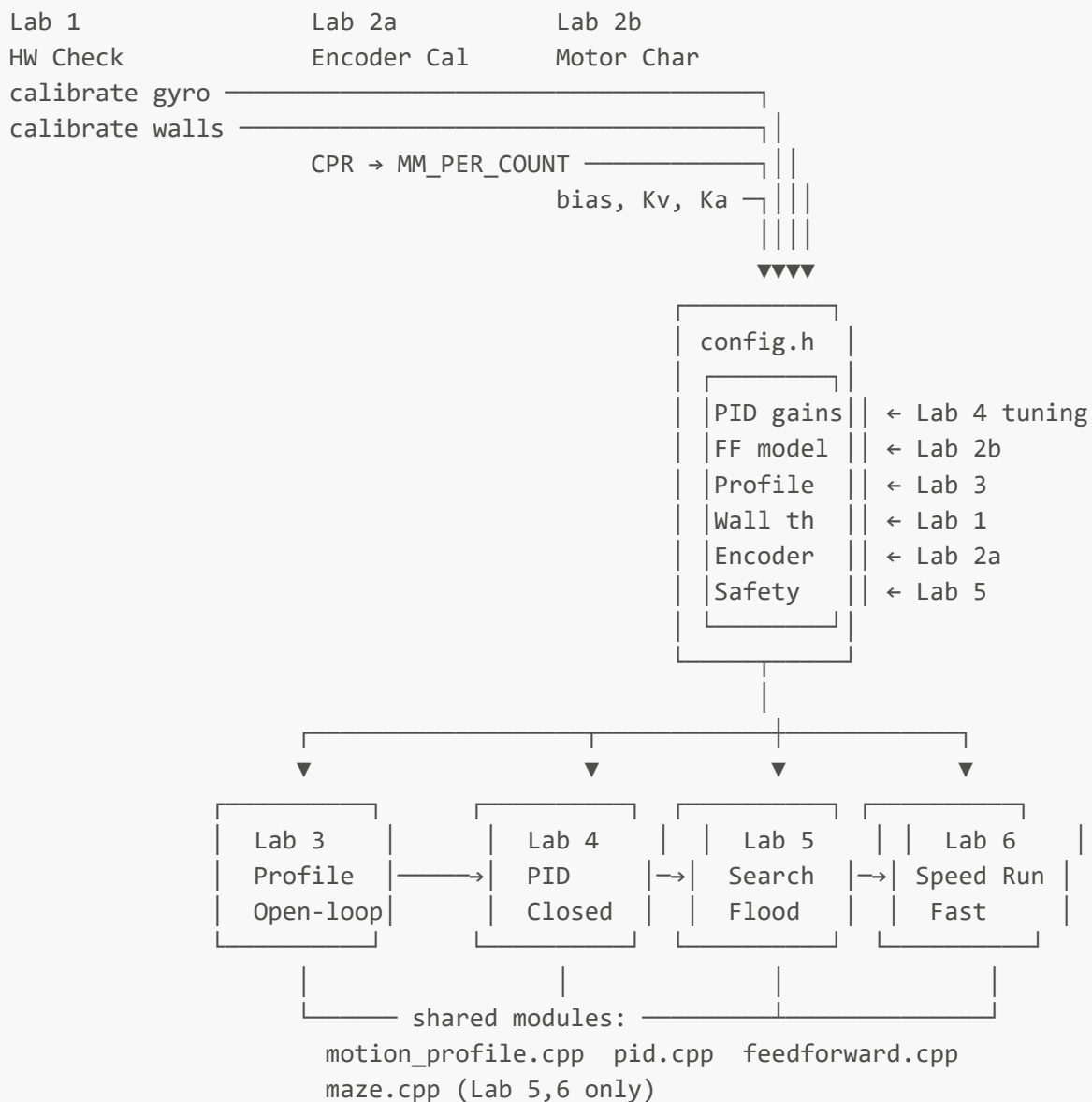
BLE notify callback



parse packet type



13. Cross-Lab Dependencies



Lab Progression (recommended order):

Lab 1 → Lab 2a → Lab 2b → Lab 3 → Lab 4 → Lab 5 → Lab 6

each lab builds on calibration/tuning from previous labs

Shared Modules ใช้โดย Lab ไหนบ้าง

Module	Lab 3	Lab 4	Lab 5	Lab 6
motion_profile.cpp	✓	✓	✓	✓
feedforward.cpp	✓	✓	✓	✓
pid.cpp	-	✓	✓	✓
maze.cpp	-	-	✓	✓

14. Config.h Parameter Map

ค่าสำคัญ จัดตาม Lab ที่ tune

Lab 1: Calibration		
WALL_HYST_FL_ON/OFF_MM	55/70	front wall detect threshold
WALL_HYST_SL_ON/OFF_MM	65/80	side wall detect threshold
WALL_DEBOUNCE_COUNT	3	consecutive samples to confirm
WALL_FRONT_DETECT_MODE	0	OR/AND/AVG/MIN logic
WALL_OFFSET_*_MM	-3/0/0/5	per-sensor calibration offset
Lab 2a: Encoder		
ENCODER_CPR	815	counts per revolution
WHEEL_DIA_MM	33.5	wheel diameter
MM_PER_COUNT	0.1291	computed: $\pi \times \text{dia} / \text{CPR}$
DEG_PER_COUNT	0.4417	computed: $360 / \text{CPR}$
Lab 2b: Motor		
FF_LINEAR_BIAS_MV	570/590	no-load voltage (L/R)
FF_LINEAR_KV	varies	mV per mm/s
FF_LINEAR_KA	varies	mV per mm/s ²
Lab 3: Profile		
CELL_SIZE_MM	180.0	maze cell dimension
SEARCH_VEL_MMPS	200	search drive speed
SEARCH_ACCEL_MMPS2	2000	search acceleration
SEARCH_DECEL_MMPS2	2500	search deceleration
Lab 4: PID		
PID_VEL_KP/KD	varies	velocity PID
PID_HEADING_KP/KI/KD	varies	heading PID
PID_WALL_KP/KD	varies	wall centering PID
Lab 5: Safety		
FRONT_BRAKE_TH1/2/3_MM	80/50/35	progressive brake thresholds
FRONT_BRAKE_CREEP_MMPS	50	minimum speed in creep zone
FRONT_ESTOP_MM	15	emergency stop distance
FRONT_ESTOP_COUNT	3	consecutive readings
FRONT_RECHECK_MM	40	post-turn wall check threshold

Lab 5: Arc Turn (Lab 6 prep)

ARC_GAP_LOOKAHEAD_MM	50	mm before center to detect gap
ARC_GAP_DEBOUNCE	2	consecutive gap readings
ARC_TURN_RADIUS_MM	40	turn arc radius

Lab 6: Speed

SPEEDRUN_VEL_0/1/2/3_MMPS	200/350/500/700	speed levels
SPEEDRUN_ACCEL_MMPS2	1000	speed run acceleration

Sensor Geometry (measured)

CELL_SIZE_MM	180.0
front sensor offset	64mm ahead of center
side sensor offset	35mm ahead of wheel, 60° angle
sensor → boundary (center)	26mm (measured 25-30mm)
sensor → far wall (center)	~100mm (measured)
side centered	33-35mm
side hugging left	L:19-20mm R:50-53mm
SHARP IR valid range	10-150mm